

1.

1.2

Pseudo

- lh t0 zahl für la t0 zahl; lw t0 0 t0; (79)
 - NOTE: aber la auch PI? (98)
- nop für addi x0, x0, 0

1.3.

Was machen die einzelnen Segmente dieses Programmes? Welche Bedeutung haben die jeweiligen Register dabei? Welche Funktion wird durch das Programm berechnet?

Lösung:

.data: Wir deklarieren und initialisieren den symbolischen Namen zahl.

.text: Im eigentlichen Programmcode lassen wir eine Schleife laufen.

Funktion der Register t0 wird mit dem Input geladen und ist der Counter für unserer Schleife t1 Enthält die Konstante 0 und dient dazu, die Schleife zu beenden t2 speichert das Ergebnis

Funktion des Programms Sei h die Funktion, die die unteren 16-bit einer binären Zahl ausgibt.

$f(x) = \sum_{n=1}^{n \leq \frac{h(x)}{16}} n$. Wobei in unserem Szenario $x = \text{Zahl}$.

Umgangssprachlich formuliert, nehmen wir das untere Halb-Wort des Inputs, teilen dies durch 16 (bit shift) und summieren alle natürlichen Zahlen bis zum Ergebnis.

1.4.

Die Eingabe kommt aus einem symbolischen Namen, der im .data-Segment deklariert wird. Dies geschieht im Speicher. TODO: .data besser verstehen

Das Register t2 interpretieren wir als Ausgabe.

NOTE: am I missing something? 'an welcher Stelle'? obere- untere bits?

1.5

NOTE: Hypothese

Ja, die Endianness spielt hier eine Rolle bei lh.

1.6

- Wir können mit x0 vergleichen, das die Konstante 0 enthält, statt t1 auf 0 zu setzen.
- NOTE: and-mask?

.data zahl: word 0x09fa00ce

.text beginning: lh t0 zahl srli t0 t0 4

middle: add t2 t2 t0 addi t0 t0 -1 bne t0 x0 middle

end: nop

—

input: 11 11 & input = input 00 & input = 00 10 & input = 10

xxxx xxxx xxxx xxxx ^^^^ ^^^^ ^^^^ — 1111 1111 1111 0000